TCSS 343 – Design and Analysis of Algorithms

Master Theorem

Master Theorem

- The methods of recursive analysis have been well studied and a helpful theorem has been proven to help us quickly find bounds for common recurrences.
  - This theorem was first introduced and proven in 1990 by Cormen, Leiserson, and Rivest in the first edition of their textbook (on this course's recommended list).
  - A simplified version of this theorem appears on page 54 of the text with a proof (suggested reading).
  - Master_theorem

- Consider a recurrence of the following form (floors and ceilings are omitted as they are not relevant to the theorem):

$$T(n) = \begin{cases} c & n < k \\ aT\left(\frac{n}{b}\right) + n^d & n \geq k \end{cases}$$

Theorem: The recurrence $T(n)$ has the following bound:

1. If $d < \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a}\right)$.
2. If $d = \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a} \log n\right)$.
3. If $d > \log_b a$ then $T(n) \in \Theta\left(n^d\right)$.

Master Theorem

$$T(n) = \begin{cases} c & n < k \\ aT\left(\frac{n}{b}\right) + n^d & n \geq k \end{cases}$$

Theorem: The recurrence $T(n)$ has the following bound:

1. If $d < \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a}\right)$.
2. If $d = \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a} \log n\right)$.
3. If $d > \log_b a$ then $T(n) \in \Theta\left(n^d\right)$.

Proof: Consider the recursion tree created by this recurrence.

Level	Size	Number	Cost Per Node	Total Cost
0	n	1	n^d	n^d
1	$\frac{n}{b}$	a	$\left(\frac{n}{b}\right)^d$	$a\left(\frac{n}{b}\right)^d$
2	$\frac{n}{b^2}$	a^2	$\left(\frac{n}{b^2}\right)^d$	$a^2\left(\frac{n}{b^2}\right)^d$
i	$\frac{n}{b^i}$	a^i	$\left(\frac{n}{b^i}\right)^d$	$a^i\left(\frac{n}{b^i}\right)^d$
k	1	a^k	c	$c \cdot a^k$

This leads to a total cost of:

$$c \cdot a^k + \sum_{i=0}^{k-1} a^i \left(\frac{n}{b^i}\right)^d$$

Master Theorem

$$T(n) = \begin{cases} c & n < k \\ aT\left(\frac{n}{b}\right) + n^d & n \geq k \end{cases}$$

Theorem: The recurrence $T(n)$ has the following bound:

1. If $d < \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a}\right)$.
2. If $d = \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a} \log n\right)$.
3. If $d > \log_b a$ then $T(n) \in \Theta\left(n^d\right)$.

Proof (continued):

$$c \cdot a^k + \sum_{i=0}^{k-1} a^i \left(\frac{n}{b^i}\right)^d$$

$$= c \cdot a^k + n^d \sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i$$

This leads to three cases depending on the ratio $\frac{a}{b^d}$:

Case 1: $a < b^d$ $(d > \log_b a)$

Then the sum is less than a constant and the bound is $\Theta\left(n^d\right)$.

Case 2: $a = b^d$ $(d = \log_b a)$

Then every term in the sum is 1 so the bound is $\Theta\left(n^d \log_b n\right)$.

Master Theorem

$$T(n) = \begin{cases} c & n < k \\ aT\left(\frac{n}{b}\right) + n^d & n \geq k \end{cases}$$

Theorem: The recurrence $T(n)$ has the following bound:

1. If $d < \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a}\right)$.
2. If $d = \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a} \log n\right)$.
3. If $d > \log_b a$ then $T(n) \in \Theta\left(n^d\right)$.

Proof (continued):

Case 3: $a > b^d \ \ (d < \log_b a)$

$$n^d \sum_{i=0}^{k-1} \left(\frac{a}{b^d}\right)^i \leq n^d \left(\frac{a}{b^d}\right)^k$$

$$= n^d \frac{a^k}{(b^k)^d} = n^d \frac{a^{\log_b n}}{(b^{\log_b n})^d}$$

$$= n^d \frac{n^{\log_b a}}{n^d} = n^{\log_b a}$$

Then the bound is $\Theta\left(n^{\log_b a}\right)$. □

Exercise: Master Theorem

$$T(n) = \begin{cases} c & n < k \\ aT\left(\frac{n}{b}\right) + n^d & n \geq k \end{cases}$$

Theorem: The recurrence $T(n)$ has the following bound:

1. If $d < \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a}\right)$.
2. If $d = \log_b a$ then $T(n) \in \Theta\left(n^{\log_b a} \log n\right)$.
3. If $d > \log_b a$ then $T(n) \in \Theta\left(n^d\right)$.

- Can we apply this theorem to these recurrences?

$$T(n) = \begin{cases} c & n \leq 1 \\ 2T\left(\frac{n}{2}\right) + dn & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n \leq 1 \\ T\left(\frac{n}{2}\right) + d & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n \leq 1 \\ T\left(\frac{n}{2}\right) + dn & n > 1 \end{cases}$$

$$T(n) = \begin{cases} c & n \leq 1 \\ 3T\left(\frac{n}{2}\right) + dn & n > 1 \end{cases}$$

TCSS 343 – Design and Analysis of Algorithms

Integer Multiplication

Multiplying Integers

- Let us return to the problem of multiplying integers:

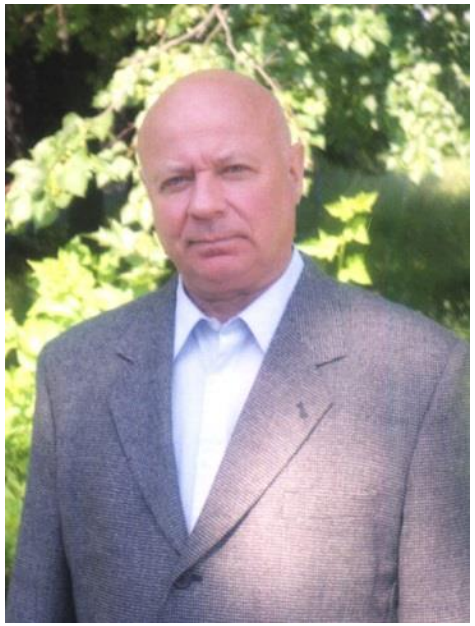
```
Integer Multiplication (MULT)
Input: Integers N and M
Output: NM an integer equal to the product of N and M
```

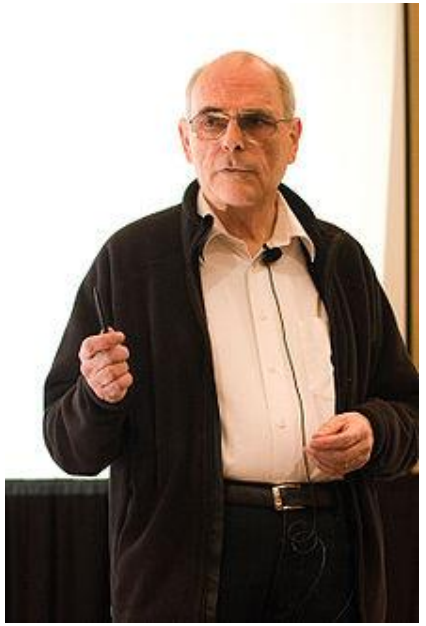
- Recall that when dealing with integers as input we use the number of bits in the integer as the measurement of the input input size.
  - To store an integer N it takes $n = \lfloor \log N \rfloor + 1$ bits.

- How many bit operations does the elementary algorithm take to multiply two n bit integers?
  - We saw in the first unit that it takes $\Theta(n^2)$ bit operations to multiply two n bit integers.
  - In 1952 Andrey Kolmogorov conjectured that this was the fastest algorithm for multiplication.
    - (Spoilers) He was wrong!

Karatsuba's Algorithm

- Divide and Conquer Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Design a recursive algorithm from the structure of the self-reduction.
  5. Analyse the runtime of the algorithm using a recurrence.
- Let's see how Anatoly Karatsuba applied this design technique in 1960 to develop an asymptotically faster integer multiplication algorithm.
  - Section 5.4 of the textbook.
  - Karatsuba_algorithm
  - Schonhage-Strassen_algorithm (extra reading)
  - Anatoly_Karatsuba
  - Volker_Strassen



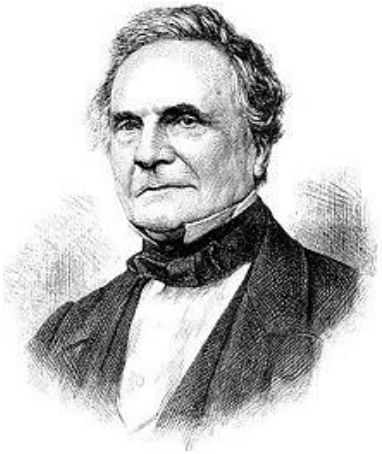


Self-Reduction

- Let $N \cdot M$ express the output of the MULT problem on input N and M.
- Can we reduce this problem to its sub-problems?
  - Hint: We can break the integers into two parts.

$$N = N_u \cdot 2^{\left\lfloor \frac{n}{2} \right\rfloor} + N_l$$

where N_l are the $\left\lfloor \frac{n}{2} \right\rfloor$ lower bits of N and N_u are the remaining $\left\lceil \frac{n}{2} \right\rceil$ upper bits of N.



- Here is a simple reduction known by Charles Babbage (see also Ada Lovelace):



$$N \cdot M = \begin{cases} 0 & \text{if } N = 0 \text{ or } M = 0 \\ M & \text{if } N = 1 \\ N & \text{if } M = 1 \\ N_u \cdot M_u 2^{2\left\lfloor \frac{n}{2} \right\rfloor} + (N_u \cdot M_l + N_l \cdot M_u) 2^{\left\lfloor \frac{n}{2} \right\rfloor} + N_l \cdot M_l & \text{if } N > 1 \text{ and } M > 1 \end{cases}$$

Self-Reduction

$$N \cdot M = \begin{cases} 0 & \text{if } N = 0 \text{ or } M = 0 \\ M & \text{if } N = 1 \\ N & \text{if } M = 1 \\ N_u \cdot M_u 2^{2\left\lfloor \frac{n}{2} \right\rfloor} + (N_u \cdot M_l + N_l \cdot M_u) 2^{\left\lfloor \frac{n}{2} \right\rfloor} + N_l \cdot M_l & \text{if } N > 1 \text{ and } M > 1 \end{cases}$$

- Can we come up with a recurrence expressing the runtime of this algorithm?
  - How many recursive calls are made by this algorithm?
  - What is the size of the recursive calls?
  - How much extra work is carried out?

$$T(n) = \begin{cases} c & n \leq 1 \\ 4T\left(\frac{n}{2}\right) + dn & n > 1 \end{cases}$$

Karatsuba's Algorithm

- Karatsuba noted that the middle coefficient could be calculated with just one additional multiplication at the cost of a few more additions (and subtractions).
  - Karatsuba discovered this version when he was 23.

- Let $P = N_u \cdot M_u, \quad Q = N_l \cdot M_l, \quad R = (N_u + N_l) \cdot (M_u + M_l).$

$$N \cdot M = \begin{cases} 0 & \text{if } N = 0 \text{ or } M = 0 \\ M & \text{if } N = 1 \\ N & \text{if } M = 1 \\ P2^{2\left\lfloor \frac{n}{2} \right\rfloor} + (R - P - Q)2^{\left\lfloor \frac{n}{2} \right\rfloor} + Q & \text{if } N > 1 \text{ and } M > 1 \end{cases}$$

- This results in the recurrence

$$T(n) = \begin{cases} c & n \leq 1 \\ 3T\left(\frac{n}{2}\right) + dn & n > 1 \end{cases}$$

- $T(n) \in \Theta\left(n^{\log_2 3}\right)$ by the Master Theorem.

Karatsuba's Algorithm

- Can we generate pseudocode for an algorithm based on this reduction?
- Let $P = N_u \cdot M_u, Q = N_l \cdot M_l, R = (N_u + N_l) \cdot (M_u + M_l)$.

$$N \cdot M = \begin{cases} 0 & \text{if } N = 0 \text{ or } M = 0 \\ M & \text{if } N = 1 \\ N & \text{if } M = 1 \\ P2^{2\left\lfloor \frac{n}{2} \right\rfloor} + (R - P - Q)2^{\left\lfloor \frac{n}{2} \right\rfloor} + Q & \text{if } N > 1 \text{ and } M > 1 \end{cases}$$

```
Multiply(N, M)
        If N = 0 or M = 0 return 0.
        If N = 1 return M.
        If N = 1 return N.
        Let P = Multiply(N_u, M_u).
        Let Q = Multiply(N_l, M_l).
        Let R = Multiply((N_u + N_l), (M_u + M_l)).
        Return P2^(2⌊n/2⌋) + (R − P − Q)2^⌊n/2⌋ + Q.
End Multiply
```

Karatsuba's Algorithm

- Example: Multiplying 20 by 35 (we should get 700).

$$N = 0001\ 0100, \qquad M = 0010\ 0011$$

- Splitting each in two we get:

$$N_u = 0001, \qquad N_l = 0100$$
$$M_u = 0010, \qquad M_l = 0011$$

- Now we can compute our products:

$$P = N_u \cdot M_u = 0010$$
$$Q = N_l \cdot M_l = 1100$$
$$R = (N_u + N_l) \cdot (M_u + M_l) = 11001$$

- And computing our difference:

$$R - P - Q = 1011$$

- Putting it all back together:

$$\begin{array}{r} 0010\ 0000\ 0000 \\ 0000\ 1011\ 0000 \\ \underline{0000\ 0000\ 1100} \\ 0010\ 1011\ 1100 \end{array}$$

TCSS 343 – Design and Analysis of Algorithms

Matrix Multiplication

Multiplying Matrices

- What is a formal statement of the matrix multiplication problem?

  Matrix Multiplication (MATRIX)
  Input: $n \times n$ matrices of integers A and B
  Output: $C = A \cdot B$ a $n \times n$ matrix.

- What is a good measurement of the input size in this problem?
  - For this problem we will consider integer additions and multiplications to be constant time.
  - We will use the size of the matrix n as the relevant input size.
- Say we are multiplying two $n \times n$ matrices. How many integer additions and multiplications will this take using our basic algorithm?

$$\begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & \ddots & \vdots \\ a_{n,1} & \cdots & a_{n,n} \end{bmatrix} \cdot \begin{bmatrix} b_{1,1} & \cdots & b_{1,n} \\ \vdots & \ddots & \vdots \\ b_{n,1} & \cdots & b_{n,n} \end{bmatrix} = \begin{bmatrix} c_{1,1} & \cdots & c_{1,n} \\ \vdots & \ddots & \vdots \\ c_{n,1} & \cdots & c_{n,n} \end{bmatrix}$$

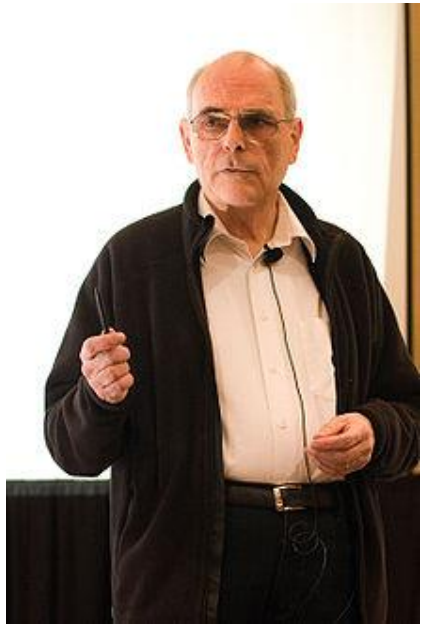
$$c_{i,j} = \sum_{k=1}^{n} a_{i,k} \cdot b_{k,j}$$

  - So each entry requires n multiplications and $n - 1$ additions.
  - There are n^2 entries.
  - This is an $O(n^3)$ algorithm.

Strassen's Algorithm

- Divide and Conquer Design
  1. State the problem as formally as possible.
  2. Identify sub-problems.
  3. Find and prove correct a self-reduction from the problem to one or more of its sub-problems.
  4. Design a recursive algorithm from the structure of the self-reduction.
  5. Analyse the runtime of the algorithm using a recurrence.

- Let's see how Volker Strassen applied this design technique in 1969 to develop an asymptotically faster matrix multiplication algorithm.
  - Section 5.4 of the textbook.
  - Strassen_algorithm
  - Coppersmith-Winograd_algorithm (extra reading)
  - Volker_Strassen



Self-Reduction

- Let $A \cdot B$ express the output of the MATRIX problem on input A and B.
- Can we reduce this problem to its sub-problems?
  - Hint: We can break the matrices into three parts.

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}$$

  - Assume that $n = 2^k$ for some $k > 0$. If this is not true we pad the initial matrices with 0s.
- Here is a simple reduction:

$$A \cdot B = \begin{cases} \begin{bmatrix} a_{1,1} \cdot b_{1,1} \end{bmatrix} & \text{if } n = 1 \\ \begin{bmatrix} A_{1,1} \cdot B_{1,1} + A_{1,2} \cdot B_{2,1} & A_{1,1} \cdot B_{1,2} + A_{1,2} \cdot B_{2,2} \\ A_{2,1} \cdot B_{1,1} + A_{2,2} \cdot B_{2,1} & A_{2,1} \cdot B_{1,2} + A_{2,2} \cdot B_{2,2} \end{bmatrix} & \text{if } n > 1 \end{cases}$$

- Can we come up with a recurrence expressing the runtime of this algorithm?

$$T(n) = \begin{cases} c & n \leq 1 \\ 8T\left(\frac{n}{2}\right) + dn^2 & n > 1 \end{cases}$$

  - By the Master Theorem this algorithm has runtime $\Theta(n^3)$.

Strassen's Algorithm

$$A \cdot B = \begin{cases} [a_{1,1} \cdot b_{1,1}] & \text{if } n = 1 \\ \begin{bmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{bmatrix} & \text{if } n > 1 \end{cases}$$

- Where:

$$C_{1,1} = A_{1,1} \cdot B_{1,1} + A_{1,2} \cdot B_{2,1}$$
$$C_{1,2} = A_{1,1} \cdot B_{1,2} + A_{1,2} \cdot B_{2,2}$$
$$C_{2,1} = A_{2,1} \cdot B_{1,1} + A_{2,2} \cdot B_{2,1}$$
$$C_{2,2} = A_{2,1} \cdot B_{1,2} + A_{2,2} \cdot B_{2,2}$$

- Strassen found a way to eliminate one multiplication at the cost of a few more additions.

$$C_{1,1} = M_1 + M_4 - M_5 + M_7$$
$$C_{1,2} = M_3 + M_5$$
$$C_{2,1} = M_2 + M_4$$
$$C_{2,2} = M_1 + M_3 - M_2 + M_6$$

- Here are Strassen's equations:

$$M_1 = (A_{1,1} + A_{2,2}) \cdot (B_{1,1} + B_{2,2})$$
$$M_2 = (A_{2,1} + A_{2,2}) \cdot B_{1,1}$$
$$M_3 = A_{1,1} \cdot (B_{1,2} - B_{2,2})$$
$$M_4 = A_{2,2} \cdot (B_{2,1} - B_{1,1})$$
$$M_5 = (A_{1,1} + A_{1,2}) \cdot B_{2,2}$$
$$M_6 = (A_{2,1} - A_{1,1}) \cdot (B_{1,1} + B_{1,2})$$
$$M_7 = (A_{1,2} - A_{2,2}) \cdot (B_{2,1} + B_{2,2})$$

Strassen's Algorithm

- Strassen's Algorithm leads to this recurrence:

$$T(n) = \begin{cases} c & n \leq 1 \\ 7T\left(\frac{n}{2}\right) + dn^2 & n > 1 \end{cases}$$

- By the Master Theorem we see that $T(n) \in O\left(n^{\log_2 7}\right)$.